

Multiversion Concurrency Control for Multiversion B-Trees

Amir Tonta
University of Marburg
Marburg, Germany
amir.tonta@mathematik.uni-marburg.de

Bernhard Seeger
University of Marburg
Marburg, Germany
seeger@informatik.uni-marburg.de

Eljas Soisalon-Soininen
Aalto University
Espoo, Finland
eljas.soisalon-soinen@aalto.fi

ABSTRACT

Multiversion concurrency control (MVCC) enables scans to read data from a committed snapshot (version), reducing conflicts with write operations compared to traditional concurrency approaches. Currently, versioned records are often managed in a B⁺-tree using version chains. However, version chains introduce overhead during scans and can still lead to conflicts between scans and writers. The multiversion B-tree (MVBT) was designed for optimal range scan performance on arbitrary versions, but has been considered impractical due to its structural complexity and, until recently, the lack of effective concurrency control. In this paper, we present the concurrent MVBT (cMVBT), a redesign of the MVBT featuring a novel concurrency control protocol that uses optimistic latches for write operations and requires no latches for range scans, while preserving all the optimality guarantees of the original MVBT. Additionally, cMVBT supports continuous garbage collection without activity spikes, seamlessly integrating free-space management. Experiments with mixed workloads derived from a standard benchmark show that the cMVBT achieves low overhead, high write throughput, and excellent range scan performance, outperforming state-of-the-art methods based on version chains.

PVLDB Reference Format:

Amir Tonta, Bernhard Seeger, and Eljas Soisalon-Soininen. Multiversion Concurrency Control for Multiversion B-Trees. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at github.com/umr-dbs/cMVBT and github.com/umr-dbs/BTree-MVCC-Version-Chains.

1 INTRODUCTION

Efficient multiversion concurrency control (MVCC) is essential for high-performance database systems [31], particularly those such as Hyper [17] that support hybrid transaction/analytical processing (HTAP) workloads [32]. By allowing queries to run on older versions (snapshots) while updates modify the most recent version, MVCC offers snapshot isolation and robust parallelism without

operational conflicts. A critical component is an index structure that supports single-row write operations (insertions, deletions, and updates) and range scans concurrently across snapshots. Designing such structures remains challenging due to the difficulty of managing multiple versions efficiently under concurrent workloads.

There are two fundamentally different approaches to indexing for version management. First, the most common approach relies on version lists attached to each key in a B⁺-tree [31]. Each version list contains entries with timestamps and values, organized chronologically, often with the most recent entry first. Single-row write operations append new versions to the front of the list, e.g., a deletion corresponds to inserting a tombstone record. Second, the other approach employs a copy-on-write B⁺-tree with path copying, as used for example in CouchDB [1]. A write operation triggers the insertion of a new path, while a scan query runs without acquiring latches by first determining the root of a committed version on which the scan is performed. However, both approaches come with their own drawbacks.

For the list-based approach, scan queries slow down as the lists grow, leading to a vicious circle of steadily declining performance, as outlined in [6]. Although clever garbage collection and more advanced list organization, such as frugal skiplists and vWeaver [18], mitigate the retrieval problem, they can incur substantial overhead for range scans for mixed workloads. Second, deletions degrade performance because keys often remain in the B⁺-tree even after they are logically deleted [16]. Third, range scans can still conflict with concurrent insertions in highly contended pages, thereby reducing query performance. These concurrency problems with high contention might be mitigated by using lightweight latches for B⁺-trees [5, 13], which provide effective mechanisms for coping with high contention in conflict-prone workloads. However, latch efficiency alone does not eliminate scalability concerns under mixed workloads.

In CoW-based approaches, write amplification is high because each write operation triggers a copy of a B⁺-tree path [1]. Consequently, garbage collection becomes more expensive and more demanding. In addition, single-write transactions must be performed sequentially. This problem can be alleviated by batching and performing batches in parallel [27]. However, this can increase latency, while the effort for space management and garbage collection remains high.

This paper takes a fundamentally different approach to version management and multiversion concurrency control. Instead of version lists and CoW approaches with path copying, we propose the concurrent multiversion B-tree (cMVBT), a concurrent index derived from the multiversion B-tree (MVBT) [4]. The MVBT is a

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

partially persistent generalization of the B⁺-tree for version management with optimal asymptotic runtime and space performance. It can be viewed as a different type of CoW B⁺-tree supporting latch-free queries, but avoiding path copying. Instead, space consumption scales linearly with the number of single-write transactions. In addition, range-scan performance asymptotically matches that of a B⁺-tree that exclusively would keep records from the target version. Thus, the MVBT has already been recognized as a candidate for the concurrency problem [18], but the authors cautioned that using the structure should be done with care because “an upsurge of new versions ... result in undesirable structure modification, thus damaging OLAP queries.” However, this paper shows that the cMVBT (the proposed extension of the MVBT with concurrency support) can be used without concern. In fact, we provide a rationale for why the cMVBT is ideally suited to implementing MVCC.

The advantages of the cMVBT can be summarized as follows. First, range-scan queries also run latch-free, and their performance is asymptotically optimal, independent of the mix of single-write transactions consisting of insertions, deletions, and updates. In general, deletions are directly supported with low overhead compared to other approaches using version lists. Garbage collection is not required to improve query performance, as is the case in [6]. Instead, on-demand garbage collection is enabled, seamlessly integrated with free-space management, resulting in substantially lower memory allocation counts. The cMVBT supports multiple write operations in parallel without batching. While our discussion in this paper is limited to using the cMVBT in main memory, all the techniques presented are applicable to external storage. In summary, our paper offers the following contributions:

- (1) The paper presents the cMVBT, a redesign of the MVBT that introduces a novel multiversion concurrency control protocol based on optimistic latching and CaS.
- (2) The cMVBT enables latch-free range-scan queries and achieves high throughput for write operations.
- (3) Write operations and range-scans preserve the theoretical efficiency bounds established in the original MVBT work.
- (4) An on-demand garbage collection mechanism is introduced, tightly integrated with free-space management.
- (5) Experiments for mixed workloads derived from a standard benchmark demonstrate that the cMVBT significantly outperforms current approaches based on version lists.

The remainder of the paper is organized as follows. Section 2 reviews related work, discusses limitations of current multiversion data structures, and sketches briefly how they are addressed in our work. In addition, it introduces the MVBT. Section 3 discusses the new page layout for the cMVBT and explains how proactive reorganizations can be enabled without incurring asymptotic performance loss. Section 4 presents the new concurrency control protocol for write operations using optimistic latching. Section 5 discusses the algorithms for range queries on snapshots. Section 6 introduces the on-demand garbage collection approach that is seamlessly integrated with free-space management. A few extensions of the cMVBT are discussed in Section 7. Section 8 presents an experimental evaluation of the cMVBT in comparison with state-of-the-art approaches. Finally, Section 9 concludes the paper.

2 RELATED WORK

This section first presents an overview of related index structures designed for multiversion concurrency control and examines their limitations. We then introduce the original MVBT without any concurrency support and its four reorganization procedures.

Multiversion concurrency control (MVCC) has become a cornerstone of modern database systems [10, 17, 24, 26], enabling concurrent transaction processing while preserving isolation and consistency. Much of the research in MVCC has focused on transaction management [8, 15, 19, 29, 31], including high-level protocols and mechanisms to coordinate transactions and enforce consistency guarantees. However, ensuring efficient concurrent access at the index level introduces additional challenges.

Traditional index structures, such as B⁺-trees, rely on in-place updates and therefore require synchronization mechanisms, such as latch coupling, to coordinate concurrent access by readers and writers [3]. To reduce synchronization overhead, [21] introduces a latch-free B⁺-tree using CAS (compare-and-swap) operations. While this approach performs well in read-dominated workloads with low update rates, it does not adequately address high-contention scenarios [30]. In contrast, our method targets mixed workloads with insertions, deletions, and updates, allowing writers to proceed without interfering with readers and reducing contention even under heavy write pressure. Nonetheless, these existing approaches can still suffer from challenges such as operation contention, potential reader failures, and starvation under intense write workloads [13]. Furthermore, ensuring consistency in such systems requires careful coordination, which introduces non-trivial overhead.

This paper addresses these limitations by introducing a redesigned tree structure tailored to MVCC systems. The key innovation is to design a concurrent append-only (copy-on-write) index derived from the MVBT [4]. Our approach ensures that:

- **Writers operate independently**, avoiding conflicts with concurrent reads. As a result, update operations do not interfere with committed data, which helps avoid latency spikes caused by contention.
- **Readers are latch-free**, operating on immutable committed data. Hence, reader queries (e.g., OLAPs) run without overhead and always succeed.
- **Reduced latch overhead in structural changes**, requiring only a single node latch for splits and at most two nodes for merges, whereas B-tree splits and merges involve multi-node coordination (parent, child, and sometimes siblings).

Our approach diverges from traditional MVCC index management with version chains [18, 31], as the cMVBT logically stores versions in separate B-trees. Physically, these versions are merged into a DAG to guarantee linear space cost.

Closely related to our approach are write-once indexes, such as the time-split B-tree [22] and copy-on-write indexes combined with path copying [1, 25, 27, 28]. However, the time-split B-tree does not support efficient deletions, whereas previous CoW approaches use path copying, which incurs high write amplification. The cMVBT overcomes these deficiencies as the amortized write cost of a single-write transaction is $O(1)$. In addition, the cMVBT allows the adoption of concepts from concurrent B-trees [13], such as optimistic latching, which has been shown to be beneficial [20].

In addition, we also integrate garbage collection (GC) directly into the index structure with minimal overhead, providing efficient free-space management and ensuring that version pruning does not hinder performance. By combining these features, our design achieves consistency in multiversion index structures while enabling efficient (OLAP) by isolating readers entirely from the effects of concurrent updates.

2.1 Review of the MVBT

The MVBT is a directed acyclic graph (DAG) that provides a compact physical representation of N immutable B⁺-trees, one for each version [4]. Whenever an update occurs on the most recent B⁺-tree, a new B⁺-tree is logically created. This update model is known as partial persistence [12], and the MVBT is an asymptotically optimal persistent search tree. Unlike other approaches [1, 27], the MVBT avoids path copying, which would result in high write amplification, superlinear storage costs $O(N \log N)$ when all N versions are maintained, and consequently, high garbage collection overhead. Instead, the MVBT adopts the same design principles as B⁺-trees, where updates typically affect only a leaf node and, only occasionally (e.g., when a node overflows), require additional nodes to be written. Therefore, for N updates, the MVBT achieves $O(N)$ space complexity and supports range scans at version i with the same asymptotic complexity as a standard B⁺-tree containing only the i -th version.

To achieve these performance guarantees, data and index entries in the MVBT nodes additionally store their associated version information. In the original MVBT design, each entry is annotated with a validity interval $[vs, ve)$, indicating that the entry is live from version vs up to ve , but not including ve . Entries corresponding to the latest version use an interval of the form $[vs, *)$. Additionally, the MVBT maintains multiple roots, with the versioned entries for these roots organized in a separate structure called *root** (see Driscoll et al. [12]). Because version intervals are disjoint and entries are appended in order, the entries in *root** can be efficiently kept in version order.

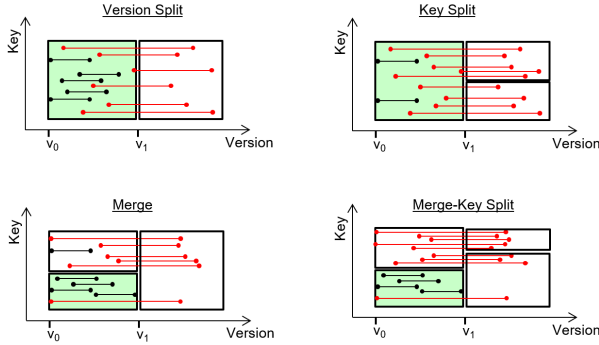


Figure 1: The Four Page Reorganizations of the MVBT

The linear storage space guarantee of the MVBT is based on two invariants. For that, let B denote the page capacity (in number of entries/records). First, the *weak-version condition* states that a linear fraction of the page capacity B belongs to the most recent version. For simplicity, we will use $d = B/5$ as the minimum number of

entries required for the most recent version. Second, the *strong-version condition* states that after a node reorganization, at least d operations on the node (insertions, deletions, updates) are required until the next reorganization can occur. These two invariants hold for all nodes except for the roots of the MVBT. For the roots, at least two entries must belong to the most recent version (assuming that more than B records are in that version).

As in standard B-trees, an operation starts at the root of the most recent version and traverses downward to a leaf. If the weak-version condition is violated or an overflow occurs, nodes are reorganized along the search path in reverse order. Below, we discuss the four reorganization operations on a leaf of the MVBT for $B = 10$ and $d = 2$, using the two-dimensional partitionings of the version-key space shown in Figure 1. Each leaf of the MVBT corresponds to a rectilinear rectangle, created at version v_0 , with reorganization occurring at version v_1 . The leaf that triggers the reorganization is emphasized (colored green). Versioned records are represented by black and red intervals, corresponding to being dead or alive at version v_1 , respectively. An insertion at version v_1 triggers a reorganization when the node contains more than B records. A reorganization always begins with a version split, in which live entries at version v_1 are copied from the original node to a new live node. If the strong-version condition is satisfied after this split, no further action is needed. Otherwise, additional reorganization steps are triggered. If the new node contains more than $8 (= 4/5 \cdot B)$ live entries, a key-split is performed first: entries are evenly distributed between two nodes using a split key from the key dimension, analogous to a split in a B-tree. If the node contains fewer than $B/5$ live entries, a merge with a live sibling is triggered. After a version split on the sibling node, the live entries from both nodes are merged into a final node, as shown in the lower-left plot of Figure 1. If the resulting node contains more than $4/5 \cdot B$ live entries, an additional key-split is performed, as illustrated in the lower-right plot of Figure 1. The two resulting live nodes then satisfy the strong-version condition. Hereafter, we use the term node reorganization to refer to version splits, merges, and key splits. Note that at most two new nodes are created during a reorganization.

The example in Figure 2 illustrates a sequence of these reorganizations applied to nodes I_0 , I_1 , and I_2 , which were alive at version 1. First, a version split occurs for node I_0 at version 5, creating node I_3 . Next, a version-key split of I_2 at version 8 creates two nodes (I_4 , I_5). At version 12, the nodes I_1 and I_4 are merged into node I_6 . Finally, a merge-key split occurs at version 16. Nodes with equal colors Figure 2 indicate that these nodes are created at the same version, i.e., share the same birth time. The labels on the y-axis denote keys: f_{min} and f_{max} are the fence keys of the domain, whereas the others $f_0, \dots, f_3$ are separators.

3 THE CONCURRENT MVBT (CMVBT)

This section first presents our preliminaries and discusses limitations of the MVBT that have to be addressed in the design of the cMVBT. Then, we outline the new node layout and show how proactive reorganizations are enabled, before detailing the management of the roots.

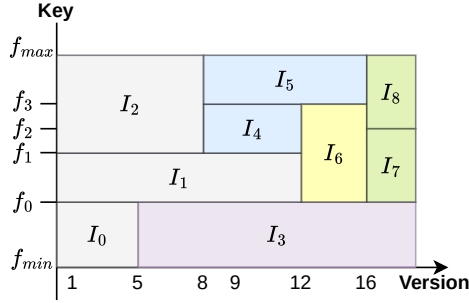


Figure 2: 2D Visualization of Entries in an Internal Node

3.1 Preliminaries and Motivating Deficiencies

Before presenting the details of our current approach, we first summarize the underlying assumptions of the paper.

- Originally, the MVBT was designed as an external data structure that stores its data elements in nodes of capacity B on external storage. Similar to the B^+ -tree, it can also be used in main memory by setting B accordingly, e.g., $B = 10$. Without loss of generality, this paper focuses on the cMVBT in main memory. However, all the MVCC techniques presented here are also applicable to the cMVBT on external storage. Note that this property does not hold for most other versioning approaches, e.g., those that employ version lists are only efficient in a main-memory setting.
- To simplify our discussions, we assume that data items are key-value pairs, where a key and a value are of fixed size. In addition, we assume that version numbers are integers with a constant size. This is not a severe limitation, as the cMVBT can use the same techniques as the B^+ -tree for variable-length data. We will discuss later how to treat variable-length keys and values in the MVBT.
- Finally, we emphasize that the paper focuses on the concurrent maintenance of the MVBT under insertions, deletions, updates and range scans for a visible version. Each operation is executed within its own transaction. Note that support for arbitrary ACID transactions is beyond the scope of this paper and will be addressed in our future work.

The original MVBT has several shortcomings in supporting concurrent operations, which the cMVBT addresses. Each shortcoming below is accompanied by a brief idea of the corresponding cMVBT solution.

- First, and perhaps surprisingly, there is no known concurrent extension of the MVBT, even though the original paper identified MVCC as a potential application. We will present a low-overhead concurrency algorithm for the cMVBT using proactive splits and optimistic latching that has recently proven highly effective [20].
- Second, the MVBT associates a version interval with each entry to track its validity and significantly restructures the nodes during insertions or updates. In contrast, the cMVBT provides a true append-only representation, which is a prerequisite for its latch-free processing of range scans.

- Third, the MVBT requires explicitly specifying the snapshot (version) for a range scan. In contrast, the cMVBT automatically determines a fresh snapshot to guarantee latch-free query execution.
- Fourth, garbage collection was not thoroughly addressed in the original MVBT, as it assumed all versions must be retained to support queries on arbitrary versions. The cMVBT instead provides a low-overhead garbage collection mechanism to reduce space overhead. Separately, as is known for the MVBT, the cMVBT also offers (asymptotically) optimal range-scan performance regardless of the amount of garbage.

3.2 Node Layout of the cMVBT

As discussed above, the cMVBT will offer a new node layout to ensure that updates are append-only. Instead of attaching version intervals to entries, an entry in the cMVBT uses only a version number. Consider the example in Figure 2 and assume that the entries of all nodes are kept in an internal node. Then, the corresponding entries are depicted in Figure 3. Every entry consists of the two fence keys of the node, the reference to the node, and the version number of its insertion, i.e., the snapshot of the operation that created the entry. Note that this results in slightly different node modifications.

$$\begin{aligned}
 I_0 &= (f_{\min}, f_0, p_0, 1) \\
 I_1 &= (f_0, f_1, p_1, 1) \\
 I_2 &= (f_1, f_{\max}, p_2, 1) \\
 I_3 &= (f_{\min}, f_0, p_3, 5) \\
 I_4 &= (f_1, f_3, p_4, 8) \\
 I_5 &= (f_3, f_{\max}, p_5, 8) \\
 I_6 &= (f_0, f_1, p_6, 12) \\
 I_7 &= (f_0, f_2, p_7, 16) \\
 I_8 &= (f_2, f_{\max}, p_8, 16)
 \end{aligned}$$

Figure 3: List of Entries in the Running Example

For an update, the cMVBT simply adds a new entry, whereas the original MVBT must also update the right boundary of a version interval. For example, the fourth entry I_3 is an update of the first entry I_0 . For deletions, we distinguish between internal nodes and leaves. For an internal node, there is no explicit deletion of an entry, but only updates can occur that cause an implicit deletion of previous entries. Consider, for example, the key-split of entry I_2 in Figure 2 that results in the creation of I_4 and I_5 . These entries cover the key range of entry I_2 , and thus, after their insertion, entry I_2 is not available in version 8 and higher. For a leaf, an entry is a triple consisting of the key, the value, and the version number. A deletion of a key is treated as an insertion of a tombstone entry that simply states that the previous key-value pair does not exist anymore from the given version. Note that the tombstone entries do not degrade efficiency because the cMVBT, like the original MVBT, guarantees the weak-version condition. Overall, each node modification creates at most two new entries appended to the end of the node, and the previous entries are not changed.

Important for concurrency is that every node maintains an 8-byte atomic value consisting of a latch bit, a counter for live entries, and a counter for dead entries. This value is used later in our optimistic latching approach to monitor the status of a node.

Figure 4 depicts the node of our running example again. The new design divides each node into two distinct sections: the committed section, which contains immutable data, and the uncommitted section. The bold red vertical line separates the committed from the uncommitted section. The uncommitted section may contain partially written data by some writer, yet it is not visible to any other parallel thread and might be undone before committing. The data in the committed section can be read without latching, which is the foundation of our latch-free scan algorithm. The latch bit of the atomic value is set to 0, indicating that no writer is currently active on that node.

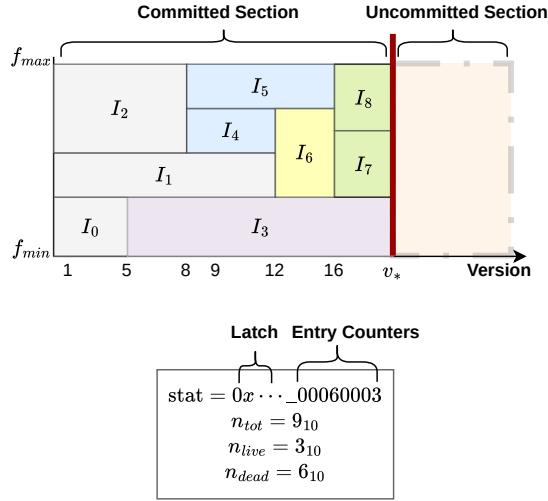


Figure 4: Committed and Uncommitted Section of an Internal Node

3.3 Proactive Reorganizations

The MVBT performs write operations (insertion, updates, and deletions) as in standard B-trees. An operation starts at the root of the most recent version and traverses downward to a leaf using the given key. If the weak-version condition is violated or an overflow occurs, nodes are reorganized along the search path in reverse order. In concurrent settings, bottom-up reorganization traversals and top-down write operations cause a high degree of conflicts. To avoid such conflicts, the cMVBT performs its reorganization steps proactively during the top-down traversal. This is a common approach used in B-trees and other search trees [14]. To introduce proactive reorganization steps for the cMVBT, we first introduce the notion of a safe node.

Definition 3.1. Let I be an internal node of the cMVBT that is alive. Let n_{tot} be the total number of entries and n_{live} be the number of live entries. I is a safe node, if

- (1) $n_{tot} < B - 1$

- (2) I is the root and $n_{live} > 1$ or I is not the root and $n_{live} > d$
- Otherwise, I is an unsafe node.

The definition can be extended for a leaf node, except that a leaf is safe if $n_{tot} < B$ and $n_{live} > d$, i.e., it allows an insertion of a new record by insertion, deletion, or update.

The new write operation starts the traversal with the current root. For each node I on the traversal path, the operation checks whether I is unsafe. If so, the corresponding reorganization is performed. If the condition $n_{tot} \geq B - 1$ for internal nodes or $n_{tot} = B$ for a leaf node is fulfilled, a reorganization is performed, as in the MVBT (version split, key split, merge-key split). All the nodes involved will be safe after the reorganization. The resulting new entries (referring to the newly created node) can then be inserted into the safe parent node. If I is the root and $n_{live} = 1$, a new root is created, and a new entry is inserted into a list containing all the roots. We will discuss the management of the list in the next section. Otherwise, I is not the root and $n_{live} = d$. Then, a merge is performed, producing safe nodes again. The new entry of the merge can again be inserted into the safe parent node. In summary, by slightly adjusting parameter settings, the proactive splits and merges of the cMVBT are possible without affecting asymptotic performance, but with only a minor impact on storage utilization and height, because nodes are split and merged slightly earlier than necessary.

3.4 Organization of the Roots

As is known from the MVBT, the cMVBT can contain multiple roots, each valid over an associated time interval. While a write operation needs only access to the root of the live version, range-scan queries use a slightly older version that is visible to them. For that reason, the cMVBT retains the roots of the older version, though we expect they are not often used, as queries seek the freshest snapshot available to them.

Thus, a structure is required for managing the roots. The simplest approach is to use a linked (or double-linked) list as illustrated in Figure 5, where there are $k+1$ roots and $k+1$ entries in the list. As

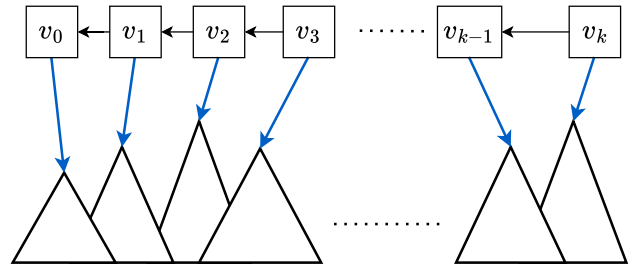


Figure 5: Root* of the cMVBT

illustrated, the heights of two adjacent trees must be either equal or differ by 1. The tree for the current root was created at version v_k and remains valid until now. Obviously, such a simple list allows appending new entries fast, but it might be slow when a query requires access to a root referenced from an element at the end of the list. For that reason, we use a frugal skip list [18], which also provides constant-time access to the most recent tree root, whereas the access to older roots is logarithmic in the number of roots.

Algorithm 1: XLatchOpt

Input: Node I , Original Status $stat$ of Node I **Output:** BOOLEAN

```

1  $stat \leftarrow stat \& \neg write$ 
2 if  $CAS(GETSTAT(I), stat, stat \mid write)$  fails then
3   return FALSE
4 return TRUE

```

4 CONCURRENT OPERATIONS

This section presents the MVCC algorithm for the cMVBt that leverages optimistic latches and optimistic latch coupling for the write operation. The first subsection provides important preliminaries for the algorithms and introduces the concept of optimistic latches. The next subsection presents the concurrency algorithms for write operations, followed by a brief introduction to read operations.

4.1 Preliminaries

In this subsection, we first introduce our version counter. We then present optimistic latching and the atomic compare-and-swap (CAS) operation in Algorithm 1. Last, we describe the function `APPEND_AND_COMMIT` called by a write operation for inserting new entries into a node Section 4.2.

4.1.1 Logical Clock for Version Ordering. Multi-Version Concurrency Control (MVCC) systems employ a global logical clock to assign monotonically increasing timestamps to operations. This mechanism establishes a consistent ordering of operations and forms the basis for visibility decisions. In our approach, the global clock is implemented as an atomic counter, ensuring thread-safe, low-overhead timestamp allocation while preserving strict monotonicity. In the following algorithms, the function `NEWVERSION()` returns the next timestamp (version).

4.1.2 Non-blocking Optimistic Latching. Algorithm 1 sketches the non-blocking latching of a node using a CAS-based approach, where the latch bit is encoded in the status value of the node. First, $stat$ is masked to remove the latch bit if set. This step is essential to ensure that the subsequent CAS operation (line 2) does not succeed based on a stale or already-latched state. In fact, it prevents multiple threads from acquiring the latch concurrently. The function `GETSTATE( $I$ )` returns the current status value for a given node I . In other words, if the CAS operation fails, it is because one of the following:

- (1) The original status value, given as parameter in `XLATCHOPT`, is outdated and differs from the current one (`GETSTAT( $I$ )`), even if no latch was present.
- (2) Node I is already latched. Thus, no other latch is allowed. Then, the actual status of I differs from the previous one given as an input parameter of `XLATCHOPT`.

Finally, our approach avoids the well-known ABA problem [9]. This is achieved because the status value includes counters that increment with every latch acquisition and release. As a result, the system never reuses the same status for different latches on the same node, thereby preventing ABA issues.

Algorithm 2: CURRENTROOT

Output: Root node r , Root's r status value r_{stat}

```

1 repeat
2    $(r, r_{stat}, root_{stat}^*) \leftarrow GETFIRST(root^*)$ 
3   if  $r$  is unsafe and  $XLATCHOPT(root^*, root_{stat}^*)$  then
4      $v \leftarrow NEWVERSION()$ 
5     if  $r$  is in overflow then
6        $(r', r'_{stat}) \leftarrow SPLITROOT(r, v)$ 
7     else
8        $(r', r'_{stat}) \leftarrow PROMOTECHILD(r, v)$ 
9      $APPEND\_AND\_COMMIT(root^*, r', v)$ 
10     $UNLATCH(root^*)$ 
11     $r \leftarrow r'$ 
12     $r_{stat} \leftarrow r'_{stat}$ 
13 until  $r$  is safe
14 return  $(r, r_{stat})$ 

```

4.1.3 APPEND_AND_COMMIT. The steps for writing data into a node and committing the operation (`APPEND_AND_COMMIT`) involve the following:

- (1) Write into the beginning of the uncommitted section.
- (2) Write the commit version into the header of the committed section.

Thus, the uncommitted section may contain partially written data by some writer (by `APPEND_AND_COMMIT`), yet it is not visible to any other thread and might be undone before committing. Consequently, readers can never access partially committed data, and there is no need to guard the access to the committed section.

The observation is that the new data becomes physically accessible immediately after it is written, before the node's latch is released. Yet this doesn't mean readers can logically access it unless the visibility check permits it. The visibility is further discussed in Section 5.1.

4.2 Write Operations

This subsection introduces the concurrent algorithm for insertions, deletions, and updates. Algorithm 4 (`WRITE TRAVERSAL`) provides a sketch of the most essential steps given key k and value v . For that matter, we first discuss Algorithm 2 for computing the live root, followed by Algorithm 3 for node reorganizations.

4.2.1 CURRENTROOT. Algorithm 2 repeatedly reads a root candidate via `GETFIRST( $root^*$ )`. This function also returns the candidate's status value, denoted by r_{stat} and $root_{stat}^*$, the status value of $root^*$. If the observed root is already safe, it is returned immediately. Otherwise, the algorithm attempts to reorganize the root by acquiring a latch on $root^*$ using the optimistic latching procedure `XLATCHOPT` (Algorithm 1).

Once the latch is successfully acquired (lines 3-4), a new version is generated via `NEWVERSION()`. Depending on the situation of the root node, either a root split (`SPLITROOT`) is performed in case of an overflow, or the last remaining active child is promoted via `PROMOTECHILD`, becoming the new root. In this case, the tree height

Algorithm 3: REPAIR

Input: Node p , Status value p_{stat} , Node c
Output: Node p , Node p 's status value p_{stat}

```
1 if  $XLATCHOPT(p, p_{stat})$  fails then
2   return  $EXIT\_AND\_RESTART()$ 
3 if  $c$  is in overflow then
4    $v \leftarrow NEWVERSION()$ 
5   if  $c$  requires version split then
6      $new\_entries \leftarrow VERSIONSPLIT(c, v)$ 
7   else
8      $new\_entries \leftarrow KEYSPLIT(c, v)$ 
9    $APPEND\_AND\_COMMIT(p, new\_entries, v)$ 
10 else
11    $(s, s_{stat}) \leftarrow SIBLING(p, c)$ 
12   if  $XLATCHOPT(s, s_{stat})$  fails then
13     return  $EXIT\_AND\_RESTART()$ 
14    $v \leftarrow NEWVERSION()$ 
15   if  $c$  and  $s$  can be merged then
16      $new\_entries \leftarrow MERGE(c, s, v)$ 
17   else
18      $new\_entries \leftarrow MERGE\_KEYSPLIT(c, s, v)$ 
19    $APPEND\_AND\_COMMIT(p, new\_entries, v)$ 
20    $UNLATCH(s)$ 
21  $UNLATCH(p)$ 
22  $p_{stat} \leftarrow LOADSTATE(p)$ 
23 return  $(p, p_{stat})$ 
```

decreases by one. The resulting node r' becomes the new root candidate.

The update is then committed atomically using the function $APPEND_AND_COMMIT$ (Section 4.1.3), which appends (r', v) to $root^*$. Finally, the latch is released, and the process repeats until a safe root is found.

4.2.2 REPAIR. Algorithm 3 lists the procedure REPAIR for maintaining the structural invariants in the tree under concurrent updates. It is invoked on a parent node p and an unsafe child c . To restore the invariants, the algorithm applies a suitable reorganization on c .

The algorithm first attempts to acquire optimistic exclusive latches on p using $XLATCHOPT$. The original status value p_{stat} of p is used to acquire the latch. Note that latching the parent node p is sufficient because an unsafe node (like c) will not be modified anymore. If this latch fails, $EXIT_AND_RESTART$ is called that behaves similar as an exception. Then, Repair is killed, all acquired latches are freed, and the entire write operation is restarted (lines 1-2).

If there is an overflow of c , a new version is generated via $NEWVERSION()$ (line 4) and the algorithm performs a suitable reorganization on c (either a $VERSIONSPLIT$ or a $KEYSPLIT$). This returns a new set of entries that the function $APPEND_AND_COMMIT$ will promote to the parent node (line 9).

Otherwise, c violates the weak-version condition. The method $SIBLING$ returns a sibling s along with its status s_{stat} (line 11). Then

it attempts to latch the sibling accordingly (line 12). If the latch on the sibling fails, the operation is restarted (line 13). Otherwise, the algorithm either merges c and s using $MERGE$, or redistributes entries between them using $MERGE_KEYSPLIT$. The resulting modifications together with version v are promoted to the parent node using $APPEND_AND_COMMIT$ (line 19). Afterward, on line 20, the latch on the sibling node s is released. Finally, the latch on the parent node p is released (line 21). The new status of the parent node is read and returned together with the updated parent node p (line 23).

4.2.3 Writetraversal. In the following, we discuss Algorithm 4. First, the algorithm calls $CURRENTROOT$ to obtain the current root and the status value (line 1). It then traverses the tree top-down from the root to the target leaf with key k . To avoid conflicting bottom-up reorganizations, the algorithm performs proactive splits as introduced in section 3.3. It checks whether the child c is safe (line 4). If not, the unsafe node is immediately repaired via the REPAIR function, and the traversal continues with the parent node p (which REPAIR returns). After exiting the loop (line 10), p refers to the target leaf that is optimistically latched via a call of $XLATCHOPT$. Finally, the method $APPEND_AND_COMMIT$ applies the changes to the leaf using the new version v .

In analogy to classical latch coupling, the number of required latches is slightly lower. In case of an unsafe node c being close to overflow, only one latch on its parent node is sufficient. A latch on c is not required, since it can no longer be modified. Once the reorganization is complete, the latch on the parent node is released. For a merge, an additional latch on the sibling of c is required. The reason is that the sibling is a safe node. Thus, we have to ensure it is not modified during the merge. All latches are released immediately after the corresponding modifications are complete, thereby minimizing contention.

4.3 Read Operations

Readers can safely access the immutable (committed) section of a node without acquiring any latch. This directly follows for historical nodes and nodes that trigger a reorganization, because these nodes can no longer be modified. If a reader accesses a live node, it first reads its status, which contains the two counters for live and dead entries (see Figure 4). In particular, a read operation can safely ignore the latch bit and thus traverse its node (i.e., the committed section) using the corresponding counters. This results in very fast and scalable reads, because:

- (1) Readers never block or retry and are independent of workload contention.
- (2) Writers only compete with other writers for the same data. There are no conflicts with read operations.

These concurrency properties of read operations are essential for the concurrent processing of queries that will be detailed in the next section.

5 SNAPSHOT QUERIES

In this section, we first outline how the cMVBT determines the freshest visible version (snapshot) for a given query in Section 5.1.

Algorithm 4: WRITE TRAVERSAL

Input: Key k , Value val

```

1  $(p, p_{stat}) \leftarrow \text{CURRENTROOT}()$ 
2 while  $p$  is an internal node do
3    $c \leftarrow \text{FINDCHILD}(p, k)$ 
4   if  $c$  is unsafe then
5      $(p', p'_{stat}) \leftarrow \text{REPAIR}(p, p_{stat}, c)$ 
6      $p_{stat} \leftarrow p'_{stat}$ 
7      $c \leftarrow p'$ 
8   end
9    $p \leftarrow c$ 
10 end
11 if  $\text{XLATCHOPT}(p, p_{stat})$  fails then
12   return EXIT_AND_RESTART()
13 end
14  $v \leftarrow \text{NEWVERSION}()$ 
15  $\text{APPEND\_AND\_COMMIT}(p, k, val, v)$ 
16  $\text{UNLATCH}(p)$ 

```

It then follows a detailed discussion of point and range queries in Section 5.2.

5.1 Snapshot Visibility

So far, the cMVBT supports only single-row write transactions, each consisting of exactly one operation (insertion, deletion, or update). In addition, it supports read transactions (point queries and range scans) on arbitrary snapshots. However, in a transactional setting, a user does not specify the snapshot. Instead, the underlying system determines the valid snapshot, ensuring the query returns the most recent data. For that, every writer thread maintains a visibility watermark corresponding to its last local commit version, initially set to a sentinel value (∞). When a writer transaction commits, it sets its visibility watermark to the commit version received from the global monotonic clock. When the reader transaction receives a query, it first computes the visible snapshot as the minimum watermark across all writer threads, and starts processing the query on that snapshot. This approach is well-established and used in other systems such as Steam [6], Hekaton [10], and Peloton [23].

To avoid stalled versions caused by inactive or read-only threads, we reset a thread's local visible commit version to a sentinel value whenever the thread aborts or transitions into a sequence of read-only transactions. During this phase, the thread does not participate in computing the globally visible watermark. Upon switching back to updating transactions, the thread resumes publishing commit versions as before, thereby overwriting the sentinel value and re-participating in determining the globally visible commit version.

5.2 Point Queries and Range Scans

This section presents the algorithms for point and range queries on a given version v . The first step for both types of queries is to determine the root, which is the entry point for version v . For that, we traverse root^* (the list of all roots) starting from the most recent entry until an entry is found whose version is lower than or equal to v .

Let us first consider the two point queries illustrated as dots in Figure 6 where the entries refer to our running example. The one query uses key k_x in version v_* , and the other uses key k_x in version 13. For the query on version v_* , it suffices to examine the live entry $I_7 = (f_0, f_2, p_7, 16)$ because $16 < v_x$ and $f_0 \leq k_x \leq f_2$. Thus, the query continues with the child p_7 that I_7 refers to. For the second query with version 13, entry I_7 is again visited, but does not qualify. Thus, the remaining entries are traversed until the first match is found. In this case, it is entry I_6 . Overall, the worst-case cost is linear in the number of elements in the node.

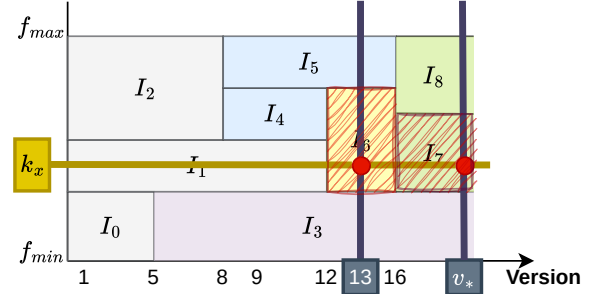


Figure 6: Two Point Queries for an Internal Node

For a range query with key range $[k_x, k_y]$ and version v , the processing of a node is more complicated because multiple entries can qualify. In the following, we present two approaches: an iterative scan algorithm and a sweep-line algorithm. The iterative scan algorithm initiates multiple point traversals. Let f_{min} and f_{max} be the fence keys of the node. The first search starts with key $k = \max(f_{min}, k_x)$. Let I be the qualifying entry and f_{min}^I, f_{max}^I the fence keys of I . While $f_{max}^I < \min(k_y, f_{max})$, the search continues with the next key $k = f_{max}^I$. Consider the range query $[k_x, k_y]$ at version v_* in Figure 7. The first point query starts with key k_x and returns entry I_7 . The next and final point query uses key f_2 and returns I_8 . The algorithm terminates because the upper fence key of I_8 exceeds k_y . Overall, the worst-case performance is quadratic in B (the capacity of the node), but as the number of qualifying entries is often observed to be $(\sqrt{B})$ [2], the practical runtime is $O(B \cdot \sqrt{B})$. In addition, the repeated scans are cache-friendly, so only the first is costly.

A different approach moves a sweep-line from right to left over the temporarily ordered sequence of entries in the node. In the sweep-line status, we maintain the current coverage of the range $[\max(f_{min}, k_x), \min(f_{max}, k_y)]$. The sweep-line moves until it reaches the state that contains all entries matching the key range at version v . In our example in Figure 7, the initial sweep-line status for the range query at version 13 contains the entries I_7 and I_8 . However, the next entry, I_6 , covers the entire query range at version 13. I_6 is the only result returned by the sweep-line method. The worst-case complexity of the method is $O(B \cdot \log B)$, but we observed that its practical performance for small B is not better than that of the iterative scan algorithm. Thus, we decided to use the iterative scan algorithm in our implementation.

Overall, the range scan algorithm performs a depth-first traversal of the tree that consists of the nodes that belong to the required

version v and applies the iterative scan algorithm to every visited internal node.

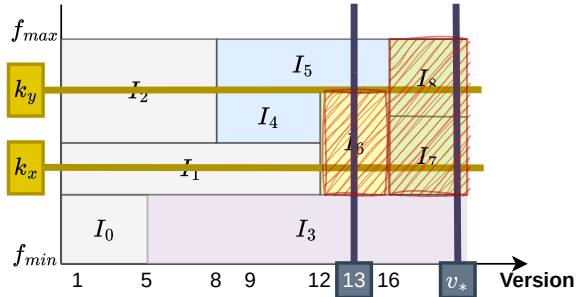


Figure 7: Two Range Queries for an Internal Node

6 GARBAGE COLLECTION

Garbage collection (GC) is essential for MVCC systems with version chains, as the length of these chains significantly impacts query performance, as discussed in [6]. Common approaches perform GC in epochs, causing activity spikes, or run GC as an additional task at query time. Recall, however, that the number of maintained versions does not impact the practical query performance of the cMVBT. The only reason for GC is to reclaim space early to avoid memory overflows. This less demanding task gives the cMVBT greater flexibility in addressing the GC problem. In the following, we first introduce our on-demand approach for GC. Then, we will discuss an optimization for overwriting entries within nodes.

6.1 On-demand GC of Nodes

The base method of the cMVBT is closely related to the one originally described for the MVBT. The idea is that nodes triggering a reorganization will become dead. These dead nodes are stored in an ordered list by death time. In the following, we refer to it as the graveyard list. Thus, the oldest nodes are in the front, and the nodes that recently died are at the end of the graveyard list. In addition, the cMVBT manages the running queries in an ordered list by start time. This list is called the active query list. Whenever a query finishes, it is removed from the active query list. If the query is the oldest, the front of the graveyard list is checked for nodes that are older than the currently active oldest query. All of these nodes can be released, thereby freeing up their storage space. This approach is also known from the methods using version chains. This could lead to a performance spike, as many nodes in the graveyard list may be affected.

By contrast, the cMVBT uses a different on-demand GC strategy, in which writer operations apply small GC steps without incurring overhead. Whenever a writer operation triggers a reorganization, the new nodes are generally allocated via the underlying memory system (which causes additional cost). However, the cMVBT uses its graveyard list as a free list. Whenever the cMVBT requests a new node, the oldest node in the graveyard index is first checked for reuse. Thus, at most two nodes are removed from the graveyard list at a time. This is a valid strategy in case the number of elements in the newest version remains stable or increases. Only if that is not

the case (which is quite unlikely), we switch to a more aggressive strategy such that an ordinary write operation (without causing a reorganization) could free space in the graveyard list. Ideally, this strategy leads to a substantial reduction in interaction with the memory system, such that many page allocations are served from the graveyard list.

6.2 Updates In-Place

Versioning is only necessary while queries are being issued. If not, it would also be possible to allow overwriting committed data records and index entries in the cMVBT. In the extreme case, the cMVBT would behave like a B⁺-tree. Consider a situation in which a key K is valid from version v_0 . If there is an update to version v_1 on K and there is no active query between v_0 and v_1 , it is safe to overwrite the old version of K . However, such an update-in-place requires more sophisticated synchronization. This is beyond the scope of this paper, and we will leave details for our future work.

7 EXTENSIONS

The cMVBT can also support common B⁺-tree extensions [20] with a moderate amount of changes, e.g., variable-length records and compression. Consider, for example, records with variable-length values. Then, we could partition a node into two parts, where the first serves as a storage area for variable-length values and the second manages the fixed part of the records (with offset and length information for the values). The versioning mechanisms operate on the fixed part of the records, ensuring that updates and versioning remain consistent without affecting the dynamic layout of the values. Despite the challenges posed by variable-length entries, node splits, and repairs, the cMVBT remains feasible. When a node overflows, the operation can compute the space required for the separator key and retain this information while checking whether the parent has sufficient space to accommodate it. If the parent cannot accommodate the new separator, it can be split proactively. This approach ensures that the standard cMVBT splitting and reorganization logic applies, even when the values are of variable size.

Techniques such as prefix compression or key delta encoding can reduce node size and improve cache efficiency. These methods are directly applicable to the MVBT, just as in traditional B⁺-trees, because compression operates on the node metadata and does not interfere with the versioning mechanism. For example, it would be possible to store the common prefix of the fence keys in the header of a node and to save this prefix for every record stored in the node. As a result, nodes can store more entries per page, improving both memory utilization and I/O efficiency.

However, the focus of this paper is not on these aspects, and we leave their concrete effects on the cMVBT for our future work.

8 EXPERIMENTAL EVALUATION

This section describes a preliminary set of experiments and discusses the most important achieved results. For that, we differentiate between results showing latency and throughput in a comparative study of the cMVBT with list-based approaches. Finally, we will discuss specific performance features of the cMVBT.

8.1 Setup

Our experiments are performed on a system equipped with an AMD EPYC 7742 64-core processor and 512 GB of main memory, running Ubuntu 22.04.1. All our implementations, including the one for the cMVBT, are written in Rust. All trees are created in main memory. In addition to the cMVBT, we consider a concurrent B⁺-tree [13] and its common extensions that support versioning. The one extension employs linked lists called Version Chains, while the other uses frugal skip lists as implemented in vWeaver [16]. The page size used by all structures is 4KB, resulting in page capacities $B = 125$ and $B = 170$ for the cMVBT and B⁺-tree, respectively.

Our data sets and workloads are generated using the standardized YCSB benchmark [7]. Our default data distribution is uniform, but some of the experiments also use Zipf distributions with parameter α . Our OLTP workloads consist of a mix of insertions, updates, and deletions. In an initial build-up phase, the first 10K operations are only insertions by default. In the second phase, we start applying a mix of updates, insertions, and deletions. A parameter, the so-called update rate, specifies the percentage of updates, whereas the number of insertions is kept equal to the number of deletions. Thus, the number of records in the most recent version will not change during the second phase. It is important to note that our experimental study considers deletions that have rarely been examined in previous studies on multiversion concurrency control. Our OLAP workload consists of queries returning the entire dataset for a given version. HTAP workloads consist of a mix of writer threads (performing insertions, updates, and deletions) and reader threads running scans. In our experiments, the default number of writer and reader threads is 32 and 16, respectively. When a thread has finished its task, it immediately receives the next one.

8.2 Results

8.2.1 Scan Latency. In our first set of experiments, we evaluate the scan performance of the three multiversion methods under OLTP workloads with varying update percentages (10%, 20%, 50%, 75%, 90%, 100%). For that, all the snapshots created by the OLTP workload are kept (no garbage collection). After processing the workload, 100K scan queries are uniformly distributed across all versions and executed sequentially.

Figure 8 depicts the average scan latency as a function of the update percentage. The 100% update workload is comparable to that in the experimental setup of [18]. For this setting, our results confirm

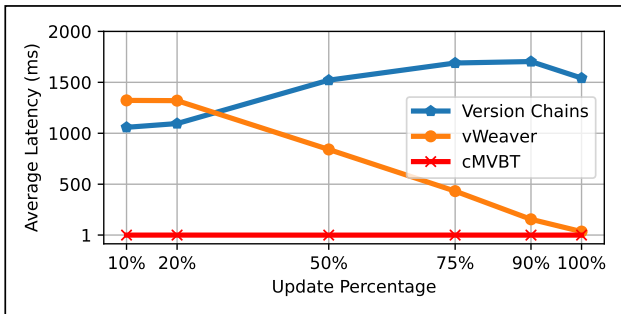


Figure 8: Scan Latency as Function of the Update Percentage

the superiority of vWeaver over Version Chains that degrade under mixed workloads for an increasing update rate and exhibit a slight slowdown-stop for update rates close to 100%. Figure 8 clearly depicts a cross-point between vWeaver and Version Chains. vWeaver is slower than Version Chains for workloads with few updates, but becomes comparable as update rates increase, and eventually superior. The reason is that version chains are very short at low update rates, so the overhead of the advanced list organization in vWeaver degrades performance. For higher update rates, this no longer has a noticeable impact.

Overall, the cMVBT is the clear winner. It is already slightly superior to vWeaver in the updates-only case, which is indeed the best case for vWeaver, as deletions do not occur. Moreover, the performance of the cMVBT is constant for all update rates and is substantially faster than Version Chains and vWeaver. These results also confirm the theoretical findings of the cMVBT that the cost of a snapshot query is determined by the result size (which is constant for all versions in this experiment) and is independent of the mix of operations in the OLTP workload. In particular, the cMVBT can handle deletions very well.

Furthermore, the results clearly show that Version Chains and vWeaver suffer from workloads with deletions because all data items accumulate in the underlying B⁺-tree. In particular, deletions cause problems in vWeaver that introduce links between adjacent skip lists. Note that the number of links between adjacent skip lists can be high, and thus, the deletion of entire lists becomes inefficient. Thus, we decided not to remove lists, with the effect that scan queries must access lists unrelated to the query. Because weaving adjacent lists only gives a benefit for very long version lists and very long lists rarely occur in our experiment (except in this one), we will not consider the weaving technique in the next experiments, but use only the pure frugal skip lists [18].

8.2.2 Concurrent Throughput. In our next experiment, we considered a concurrent HTAP workload with 32 writer threads and 16 reader threads. Instead of latency, we report throughput, which is more commonly used in concurrent evaluations. The writer threads used the same settings as in our previous experiments: an initial phase of 10K insertions followed by workloads of 1M operations with varying update rates. The 16 reader threads run snapshot scans on the freshest visible version as outlined in Section 5.1. All reader threads were terminated immediately upon completion of all the write operations.

Figure 9 provides two results plots for this setup. The upper one shows the throughput in snapshot scans per second, and the lower one the throughput in OLTP operations per second. As mentioned above, vWeaver uses pure frugal skip lists, but no links among adjacent lists. vWeaver exhibits slightly lower OLTP throughput than Version Chains due to additional overhead from managing version lists. Moreover, there is no advantage of vWeaver for scan throughput, because queries run on fresh snapshots that often require only the first elements in a list. The cMVBT consistently outperforms both vWeaver and Version Chains in OLTP throughput and scan throughput. The reason for its high scan throughput is its latch-free query processing. The performance advantage in OLTP throughput is due to the cMVBT having no conflicts with reader threads.

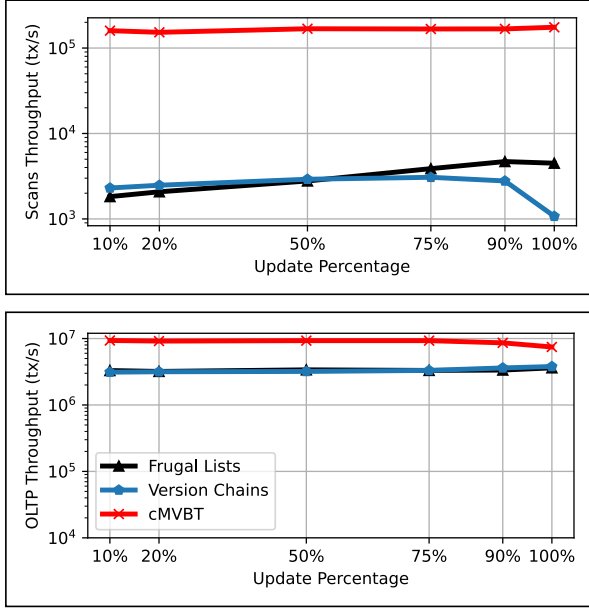


Figure 9: Throughput in Concurrent OLTP Workloads Without GC

We also conducted the same experiment with GC enabled, as shown in Figure 10. We equipped the B-tree with a garbage-collection technique that runs during reorganizations, i.e., page splits and merges remove deleted keys, ultimately preventing unsustained growth. Moreover, consistent with the results in Figure 10 and Figure 9, both Version Chains and vWeaver remain slower than cMVBT

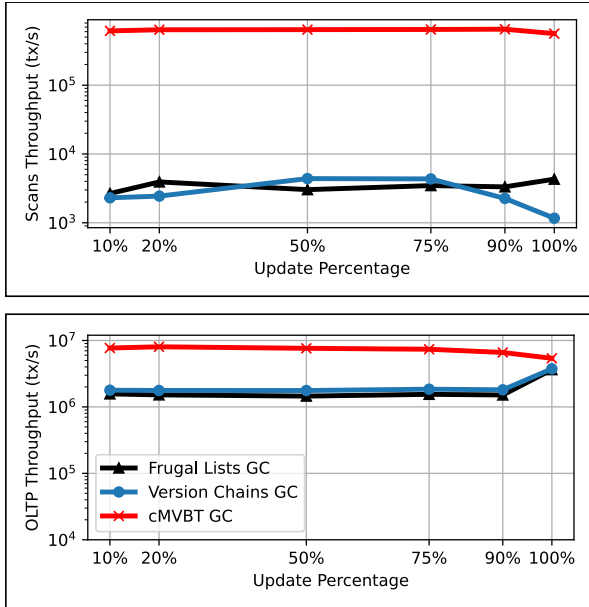


Figure 10: Throughput in Concurrent OLTP Workloads with GC

in scan and OLTP throughput, regardless of whether garbage collection (GC) is enabled or not. This suggests that the underlying data structure is a dominant contributor to query throughput. In contrast, cMVBT organizes data by version into contiguous blocks. Because readers access only immutable data, synchronization overhead is low, and the block-oriented layout yields significantly improved cache efficiency.

In addition, the primary role of GC in the cMVBT is to maintain stable memory consumption by retaining only relevant data and preventing unbounded memory growth. The system continuously reclaims obsolete versions, ensuring that memory usage remains sustainable even under long-running workloads. Moreover, reclaimed nodes are recycled internally, which reduces the frequency of memory allocation and deallocation. This reuse not only lowers memory management overhead but also improves cache behavior and overall execution efficiency.

8.2.3 Scalability of Throughput. Next, we examined the scalability of our concurrency control strategy as a function of the number of threads for both OLTP and OLAP operations in an HTAP setting (60% update rate). Figure 11 displays the results for the cMVBT and B⁺-tree with frugal skiplists. OLTP throughput grows rapidly for the cMVBT, primarily because there are no conflicts with concurrently running scan operations. In general, root-to-leaf traversals are latch-free and require no validation of optimistic latches. Only leaf nodes are briefly latched during writes. To the contrary, OLTP throughput does not scale well for frugal lists, although the B⁺-tree also employs optimistic latching. However, the crucial difference to the cMVBT is that the optimistic latches in a B⁺-tree require a validation process [20] with repeated atomic version checks (e.g., acquire load that ensures memory operation ordering). This synchronization overhead limits the scalability under mixed OLTP/OLAP contention. Read

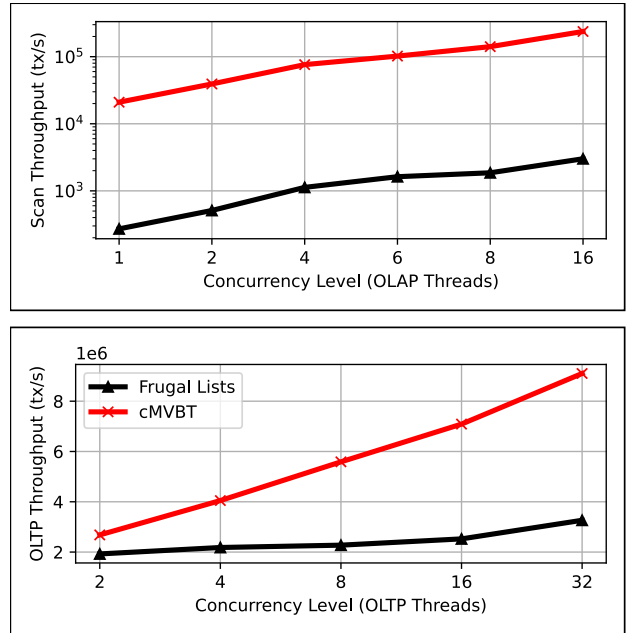


Figure 11: Scalability of Throughput

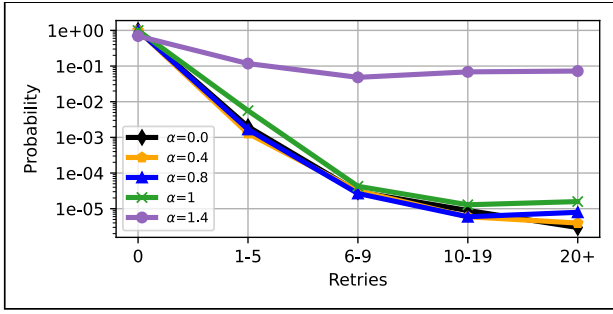


Figure 12: Probability of retry groups for various Zipf distributions

operations, by contrast, scale without introducing contention in the cMVBT but are bounded by memory bandwidth. Frugal lists, on the other hand, show a similar scaling trend for OLAP workloads, but additional latch contention reduces the throughput. Overall, the throughput is two orders of magnitude lower than that of cMVBT, independent of the number of threads. This is consistent with the results of our previous experiments. For that reason, we did not report results for version chains that perform similarly to those of frugal skiplists.

8.2.4 Specific Features of the cMVBT. This section provides important insights about the cMVBT.

First, we examined the overhead of optimistic latches in an OLTP experiment with 1M insertions. The results in Figure 12 show the probability of retries (logarithmic scale) as a function of five retry groups (0, 1-5, 6-9, 10-19, 20+) for different kinds of Zipf distributions ($\alpha = 0, 0.4, \dots, 1.4$). For example, retry groups 0 and 1-5 mean that no retries and 1 to 5 retries are performed, respectively, until the operation succeeds. The graphs show that the probability of a retry is low, except for $\alpha = 1.4$. In general, the cMVBT can cope with highly skewed data. Other studies on optimistic latching [11] have not examined higher values for α as in our experiments.

Second, we examined the value of the adaptive GC. Figure 13 displays the number of node allocations in the MVBT where the upper curve shows the number of nodes reused and the lower curve shows the nodes that are allocated from the memory manager. Results are again given as a function of the update rate. Overall, the results show that nearly all node allocation are taken from the GC list, whereas only 1% are from the memory manager. In case of updates only, the MVBT only applies version splits, and thus, the reuse is almost 100%. One reason for this result are also due to our workloads that keep the number of live records very stable over time.

9 CONCLUSION AND FUTURE WORK

This paper studied the concurrency problem for an HTAP workload with range scans and write operations, each of which consists of a single operation (insertion, deletion, and update), also known as single-row write transactions. To address this problem, we presented the cMVBT, a concurrent multiversion index structure designed to deliver high performance for these HTAP workloads. Rather than using version chains, the cMVBT employs an extended multiversion B-tree that offers the same asymptotic performance

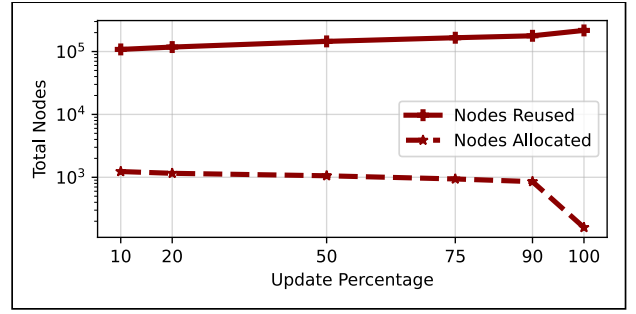


Figure 13: Nodes Reuse vs. Nodes Allocations in an OLTP Workload via GC

for every snapshot as an ordinary B-tree. In particular, the cMVBT has substantially redesigned the MVBT, introducing a new page layout, proactive splits, and most importantly, a novel multiversion concurrency control protocol that combines CAS and optimistic latching. As a unique result, range scans run latch-free and without having conflicts with write operations. This is especially beneficial when using optimistic latching, as it reduces the number of failed write operations that must be restarted. Scan operations never fail because they run only on immutable data.

The results of an experimental evaluation demonstrate the consistent superiority of the cMVBT over B-trees with version chains for both scan and OLTP throughput across various workloads, where write operations comprise insertions, deletions, and updates. The lowest superiority over an advanced version-list approach is observed for updates only. For workloads including insertions and deletions, the differences can be orders of magnitude.

Overall, we conclude that the cMVBT offers high concurrency on an efficient multiversion index structure, making it well-suited to HTAP workloads that require both high transactional throughput and efficient analytical scans.

In our ongoing work, we extend the cMVBT to manage data stored on external storage and integrate it into a storage engine that supports multiversion transactions comprising multiple operations. Furthermore, we plan to examine compression techniques and a more advanced node design to improve traversals.

REFERENCES

- [1] J. Chris Anderson, Jan Lehnardt, and Noah Slater. 2010. *CouchDB - The Definitive Guide: Time to Relax*. O'Reilly. <http://www.oreilly.de/catalog/9780596155896/index.html>
- [2] Lars Arge, Octavian Procopiuc, Sridhar Ramaswamy, Torsten Suel, and Jeffrey Scott Vitter. 1998. Scalable sweeping-based spatial join. In *VLDB*, Vol. 98. 570–581.
- [3] Rudolf Bayer and Mario Schkolnick. 1977. Concurrency of operations on B-trees. *Acta informatica* 9, 1 (1977), 1–21.
- [4] Bruno Becker, Stephan Gschwind, Thomas Ohler, Bernhard Seeger, and Peter Widmayer. 1996. An asymptotically optimal multiversion B-tree. *The VLDB Journal* 5 (1996), 264–275.
- [5] Jan Böttcher, Viktor Leis, Jana Giceva, Thomas Neumann, and Alfons Kemper. 2020. Scalable and robust latches for database systems. In *Proceedings of the 16th International Workshop on Data Management on New Hardware*. 1–8.
- [6] Jan Böttcher, Viktor Leis, Thomas Neumann, and Alfons Kemper. 2019. Scalable garbage collection for in-memory MVCC systems. *Proceedings of the VLDB Endowment* 13, 2 (2019), 128–141.
- [7] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.

- [8] Mohammad Dashti, Sachin Basil John, Amir Shaikhha, and Christoph Koch. 2016. Repairing conflicts among MVCC transactions. *arXiv preprint arXiv:1603.00542* (2016).
- [9] Damian Dechev, Peter Pirkelbauer, and Bjarne Stroustrup. 2010. Understanding and effectively preventing the ABA problem in descriptor-based lock-free designs. In *2010 13th IEEE international symposium on object/component/service-oriented real-time distributed computing*. IEEE, 185–192.
- [10] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server’s memory-optimized OLTP engine. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 1243–1254.
- [11] Bailu Ding, Lucja Kot, and Johannes Gehrke. 2018. Improving optimistic concurrency control through transaction batching and operation reordering. *Proceedings of the VLDB Endowment* 12, 2 (2018), 169–182.
- [12] James R Driscoll, Neil Sarnak, Daniel Dominic Sleator, and Robert Endre Tarjan. 1986. Making data structures persistent. In *Proceedings of the eighteenth annual ACM symposium on Theory of computing*. 109–121.
- [13] Amir El-Shaikh, Bernhard Seeger, and Eljas Soisalon-Soininen. 2024. Lightweight Latches for B-Trees to Cope with High Contention. In *International Conference on Database and Expert Systems Applications*. Springer, 217–232.
- [14] Leo J Guibas and Robert Sedgwick. 1978. A dichromatic framework for balanced trees. In *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. IEEE, 8–21.
- [15] Tuukka Haapasalo, Ibrahim Jaluta, Bernhard Seeger, Seppo Sippu, and Eljas Soisalon-Soininen. 2009. Transactions on the multiversion B+-tree. In *Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology*. 1064–1075.
- [16] Theodore Johnson and Dennis Sasha. 1993. The performance of current B-tree algorithms. *ACM Transactions on Database Systems (TODS)* 18, 1 (1993), 51–101.
- [17] Alfons Kemper and Thomas Neumann. 2011. HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots. In *2011 IEEE 27th International Conference on Data Engineering*. IEEE, 195–206.
- [18] Jongbin Kim, Kihwang Kim, Hyunsoo Cho, Jaeseon Yu, Sooyong Kang, and Hyungsoo Jung. 2021. Rethink the scan in mvcc databases. In *Proceedings of the 2021 International Conference on Management of Data*. 938–950.
- [19] Jongbin Kim, Jaeseon Yu, Jaechan Ahn, Sooyong Kang, and Hyungsoo Jung. 2022. Diva: Making mvcc systems http-friendly. In *Proceedings of the 2022 International Conference on Management of Data*. 49–64.
- [20] Viktor Leis, Michael Haubenschild, and Thomas Neumann. 2019. Optimistic Lock Coupling: A Scalable and Efficient General-Purpose Synchronization Method. *IEEE Data Eng. Bull.* 42, 1 (2019), 73–84.
- [21] Justin J Levandoski, David B Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for new hardware platforms. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 302–313.
- [22] David Lomet and Betty Salzberg. 1989. Access methods for multiversion data. *ACM SIGMOD Record* 18, 2 (1989), 315–324.
- [23] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C Mowry, Matthew Perron, Ian Quah, et al. 2017. Self-Driving Database Management Systems. In *CIDR*, Vol. 4. 1.
- [24] Andrew Pavlo and Matthew Aslett. 2016. What’s really new with NewSQL? *ACM Sigmod Record* 45, 2 (2016), 45–55.
- [25] Benjamin Sowell, Wojciech M. Golab, and Mehul A. Shah. 2012. Minuet: A Scalable Distributed Multiversion B-Tree. *Proceedings of the VLDB Endowment* 5, 9 (2012), 884–895.
- [26] Michael Stonebraker and Lawrence A Rowe. 1986. The design of Postgres. *ACM Sigmod Record* 15, 2 (1986), 340–355.
- [27] Yihan Sun, Guy E. Blelloch, Wan Shen Lim, and Andrew Pavlo. 2019. On Supporting Efficient Snapshot Isolation for Hybrid Workloads with Multi-Versioned Indexes. *Proc. VLDB Endow.* 13, 2 (2019), 211–225. <https://doi.org/10.14778/3364324.3364334>
- [28] Andy Twigg, Andrew Bye, Grzegorz Miłoś, Tim Moreton, John Wilkes, and Tom Wilkie. 2011. Stratified B-trees and Versioned Dictionaries. In *Proceedings of the 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '11)*. USENIX Association, Portland, OR, USA, 10–10.
- [29] Jochen Van den Bercken and Bernhard Seeger. 1996. *Query processing techniques for multiversion access methods*. Citeseer.
- [30] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G Andersen. 2018. Building a bw-tree takes more than just buzz words. In *Proceedings of the 2018 International Conference on Management of Data*. 473–488.
- [31] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An empirical evaluation of in-memory multi-version concurrency control. *Proceedings of the VLDB Endowment* 10, 7 (2017), 781–792.
- [32] Chao Zhang, Guoliang Li, Jintao Zhang, Xinning Zhang, and Jianhua Feng. 2024. Htap databases: A survey. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 6410–6429.